

GENAI

INTERVIEW QUESTION BANK

(Beginner to Production)

What's inside?

- ✓ Covers fundamentals to advanced & production-level topics
- ✓ Real-world scenarios, coding, system design & behavioral Qs
 - ✓ Essential for GenAI / LLM / RAG / Agent interviews
 - ✓ Handpicked questions from industry best practices
- ✓ Perfect for engineers, builders & aspiring AI professionals

"The best way to predict the future is to build it."
— Build. Learn. Ship.

Compiled & shared by

Thirumurugan Munusamy

AI Technical Architect | PhD Researcher (AI & Data Science)
SRM Institute of Science & Technology, Chennai, India

Email: thirvyrrion369@gmail.com

Phone: +91 98436 23037

LinkedIn: <https://www.linkedin.com/in/thirumurugan-munusamy-77865188/>

GitHub: <https://github.com/Thiru1988Mtech>

Coursera: <https://www.coursera.org/user/1ac81923769b9673c3381fd91937423e>

Generative AI | LLMs | RAG | Agentic AI | MLOps | AWS | Azure | GCP
14+ Years in AI, Data, Integration & Enterprise Systems

Contents

Section 1 — BEGINNER (Q1 – Q10)

Section 2 — INTERMEDIATE (Q11 – Q20)

Section 3 — ADVANCED (Q21 – Q30)

Section 4 — SCENARIO-BASED (Q31 – Q35)

Section 5 — CODING (Q36 – Q38)

Section 6 — BEHAVIORAL (Q39 – Q42)

Section 7 — MODEL SELECTION & INFRASTRUCTURE (Q43 – Q58)

Section 8 — ADDITIONAL INTERMEDIATE (Q59 – Q74)

Section 9 — SYSTEM DESIGN SHORT-ANSWER (Q75 – Q80)

SECTION 1 — BEGINNER (Q1 – Q10)

Q1. What is a Large Language Model and how is it different from traditional ML models?

LLMs are deep learning models (usually transformer-based) trained on vast amounts of text to understand and generate human-like language. Traditional ML models are task-specific, use hand-crafted features, and need more labeled data.

Q2. What is tokenization and why does it matter?

Tokenization converts text into smaller units (tokens) that models can understand. It affects context length, cost, performance, and how text is represented by the model.

Example: "Hello, world!"

Hello | , | world | !

Q3. What is an embedding and what is it used for?

An embedding is a dense vector representation of text (or other data) in a high-dimensional space. Used for semantic search, clustering, recommendations, similarity, and as input to many NLP systems.

Text → [0.12, -0.45, 0.33, ...] (Embedding vector)

Q4. What is RAG and what problems does it solve?

RAG (Retrieval-Augmented Generation) retrieves relevant documents from a knowledge base and uses them as context for the LLM. Solves problems like outdated knowledge, hallucinations, and lack of domain-specific information.

Query → Retriever (Vector DB) → LLM → Answer
(Retriever pulls Top-K docs into the LLM's context)

Q5. What is a vector database and how is it different from a traditional database?

A vector DB stores and indexes embeddings for similarity search (e.g. cosine similarity). Traditional DBs store structured data (rows/columns) and are optimized for exact matches and transactions.

Vector DB	Traditional DB
Finds similar vectors	Finds exact matches

Q6. What is prompt engineering and why does it matter?

Prompt engineering is the art of crafting inputs (prompts) to get better, more reliable, and more relevant outputs from an LLM.

Q7. What is the context window and why does it matter?

The context window is the maximum number of tokens the model can "see" in one request (prompt + response). It limits how much information you can provide and affects cost and performance.

Context Window = System Prompt + User Prompt + Response
(Limited by the model — fixed capacity)

Q8. What is hallucination and what causes it?

Hallucination is when the model generates confident but false or unsupported information. Causes: missing context, ambiguous questions, over-generalization, or training-data limitations.

Model says: "The CEO of Apple in 2021 was ____" (incorrect / unverifiable info)

Q9. Explain the difference between fine-tuning, RAG, and prompt engineering. When do you use each?

- Prompt Engineering: No training needed. Use for general tasks, quick iteration.
- RAG: No model training. Use when you need up-to-date or private/domain data.
- Fine-tuning: Train the model on your data. Use for specialized behavior, format control, or when the model needs to "learn" your domain deeply.

Method	Use When...
Prompt Eng.	General purpose, quick wins
RAG	Need external / up-to-date knowledge
Fine-tuning	Need the model to learn domain-specific patterns

Q10. What is the difference between encoder-only, decoder-only, and encoder-decoder transformer architectures?

Encoder-Only (e.g. BERT)	Decoder-Only (e.g. GPT)	Encoder-Decoder (e.g. T5)
Understands input text. Used for classification, search, embeddings.	Generates text autoregressively (one token at a time). Used for generation, chat, code.	Encoder understands input, decoder generates output. Used for translation, summarization.

Note: Most LLMs you use daily (ChatGPT, Claude, Llama, Mistral) are Decoder-Only.

SECTION 2 — INTERMEDIATE (Q11 – Q20)

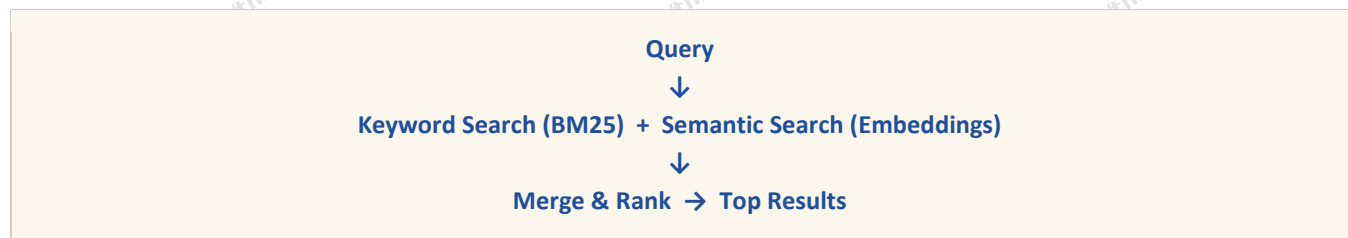
Q11. Walk me through the failure modes of a naive RAG system and how you fix each one.

Failure Mode	How to Fix
Poor retrieval (irrelevant docs)	Better embeddings, reranker, hybrid search
Missing context (relevant docs not retrieved)	Increase recall, query rewrite, hybrid search
Bad chunking (too big / small, splits meaning)	Semantic chunking, smaller overlapping chunks
Hallucinations / unsupported answers	Stricter prompting, include citations, confidence check
High latency	Caching, smaller model, retrieve less
High cost	Reduce context, compression, cheaper model
Data leakage / wrong source control	Proper authZ, metadata filters, tenant isolation

Q12. What is hybrid search and when is it better than pure semantic search?

Hybrid search combines keyword search (BM25) + semantic (vector) search and merges the results.

- Better when: Query has exact terms / codes / names / numbers
- Spelling variations
- Acronyms / shorthand
- Mix of lexical + conceptual meaning



Q13. Explain the difference between LoRA and QLoRA. When would you choose one over the other?

LoRA	QLoRA
Low-Rank Adaptation	Quantized LoRA
Adds trainable rank-decomposition matrices to attention/FFN layers	Base model loaded in 4-bit with double quantization
Base model kept in full precision (e.g. FP16/BF16)	LoRA adapters are trainable (usually FP16)
Higher quality, higher memory	Much lower memory, slightly slower training

Choose: LoRA when you have enough GPU memory and want max quality. QLoRA when you have limited GPU memory and need cost-efficient training.

Q14. What are the RAGAS metrics and what does each one tell you about your RAG system?

Metric	What it tells you
Faithfulness	Are the answers factually consistent with the retrieved context?
Answer Relevance	How relevant is the answer to the question?
Context Precision	How much of the retrieved context is actually relevant?
Context Recall	How much of the relevant information was retrieved?
Answer Correctness	Is the final answer correct (vs. ground truth)?

Q15. What is the ReAct agent pattern and how does it work?

ReAct = Reasoning + Acting. The agent alternates between thinking (reasoning) and taking actions (using tools), observing the results in between.

Thought (Reason) → Action (Use Tool) → Observation (Result)
→ Thought (Reason) → ... → Final Answer

Works as a loop until the agent has enough information to answer.

Q16. What is LLM-as-judge and what are its known biases?

LLM-as-judge uses an LLM to evaluate outputs (e.g. relevance, helpfulness). It's useful but not perfect.

- Known biases:
 - Self-preference bias (favors similar style)
 - Position bias (prefers first/last option)
 - Verbosity bias (prefers longer answers)
 - Lack of real-world grounding

Q17. What is the OWASP LLM Top 10 and which are most relevant to an engineer building a production RAG system?

1. Prompt Injection
2. Sensitive Info Disclosure
3. Supply Chain Vulnerabilities
4. Data & Model Poisoning
5. Improper Output Handling

6. Excessive Agency
7. System Prompt Leakage
8. Vector & Embedding Weaknesses
9. Misinformation
10. Unbounded Consumption

Most relevant for RAG engineers: #1 Prompt Injection, #2 Sensitive Info Disclosure, #4 Data & Model Poisoning, #8 Vector & Embedding Weaknesses — they directly impact retrieval, content safety, and multi-tenant data integrity.

Q18. What is MCP (Model Context Protocol) and what problem does it solve?

MCP Host (LLM App) ⇔ Standardized protocol for context
⇔ Data Sources: Tools/APIs, Vector DBs, File Systems

Problem it solves: different integrations lead to fragmented code. MCP standardizes how tools, contexts, and resources are exposed to models — easier, safer, and reusable.

Q19. What is prompt compression and how does it reduce cost without sacrificing quality?

Prompt compression reduces token count while keeping the essential meaning.

- Summarization of long docs / chat history
- Removing redundant context (keep key facts only)
- Semantic filtering — structured formats (tables, JSON)

Result: Fewer input tokens reduce cost while preserving the signal.

Q20. How does vLLM enable high-throughput LLM serving?

- PagedAttention: efficient memory management for long contexts
- Continuous Batching: groups incoming requests dynamically to keep GPUs fully utilized
- Optimized KV Cache: better cache reuse & memory efficiency
- Tensor Parallelism & Multi-GPU: scales across GPUs with minimal overhead
- OpenAI-compatible API: drop-in replacement

Requests → vLLM Engine (Continuous Batching) → Higher Throughput / Lower Latency

Key takeaway: Build systems that are relevant, reliable, safe, and efficient.

SECTION 3 — ADVANCED (Q21 – Q30)

Q21. Your production RAG system is working well on simple queries but failing on multi-hop questions. What is the architecture change?

Use a Multi-Hop / Decomposition + Iterative Retrieval architecture.

Query → Query Decomposition (LLM) → Sub-queries (1, 2, ...)
→ Retrieve for each sub-query (parallel) → Synthesis/Reason (LLM) → Answer

- Break the question into sub-questions
- Retrieve iteratively — each hop uses prior context
- Maintain a working memory / scratchpad
- Optionally build a knowledge graph or use GraphRAG for entity hops

Q22. Describe the lost-in-the-middle problem and how you engineer around it.

Lost-in-the-Middle: in long contexts, LLMs tend to pay less attention to information in the middle and focus more on the beginning and end.

Risk: Important context placed in the middle gets ignored.

Beginning (High attention) — Middle (Low attention) — End (High attention)

- Put the most important info at the beginning and end
- Use contextual compression / summarization to shrink the middle
- Re-order chunks by relevance (most → least)
- Keep context-window utilization moderate ($\leq 70-80\%$)

Q23. How do you design an LLM application for strict latency requirements (sub-800ms response)?

Technique	How
Use faster models	Choose small / distilled models (7B, 8x7B MoE) or optimized APIs (gpt-4o-mini, Claude Haiku, etc.)
Reduce tokens	Short prompts, summarize context, compress docs, use few-shot sparingly
Optimize retrieval	Tight top-k (3–5), use embedding filters & metadata filters to cut noise
Parallelize	Run retrieval, reranking, tool calls in parallel where possible
Cache aggressively	Semantic cache for queries / responses
Edge / regional inference	Use nearby regions or edge inference to reduce network latency
Streaming + early return	Stream tokens back, return partial results quickly
Continuously profile	Measure every stage. Optimize the slowest component

Target budget (sub-800ms): Retrieval < 150ms, Rerank < 100ms, LLM < 400ms, Overheads < 150ms.

Q24. Explain how you would prevent an AI agent from entering an infinite loop in production.

Control	What it does
Max steps / iterations	Set a hard limit on the number of reasoning or tool-use steps
Time budget	Enforce an overall timeout for the task
Loop detection	Track repeated states, actions, or tool calls
Action deduplication	Prevent identical actions being executed repeatedly
Guardrails / rules	Define allowed actions and stop conditions
Final answer requirement	Force the agent to produce a final answer after N steps
Human-in-the-loop fallback	Escalate to a human when stuck

Q25. How would you handle multi-tenancy in a RAG system where different users must only access their own documents?

Control	How
Tenant isolation	Store tenant_id with every document and embedding
Filtered retrieval	Always filter by tenant_id (metadata filter) during retrieval
Separate indexes (option)	Per-tenant index or namespace for strong isolation
Row-level security	Enforce RLS at the DB / storage layer for docs & metadata
Audit & monitoring	Log access per tenant, monitor anomalies

Golden rule: Never rely only on application logic — enforce isolation at the data / index layer.

Q26. A user's query gets routed to your RAG system but the query is ambiguous. What do you do?

Detect Ambiguity → Clarify with User → Proceed with Clarified Query

- Detect ambiguity: low-confidence retrieval, multiple possible intents, vague / underspecified
- Clarify with user: ask targeted questions or provide options ("Do you mean ... or ...?", "Which time period are you referring to?")

If clarification fails: Fall back to a safe default (general answer) or route to human support.

Q27. What is the difference between temperature, top-p, and top-k sampling? When do you adjust each?

- Temperature (T): controls randomness by scaling logits before sampling.
 - Low (0–0.3): more deterministic, factual answers
 - High (0.8–1.2+): more creative, diverse outputs
- Top-p (nucleus sampling): samples from the smallest set of tokens whose cumulative probability $\geq p$.
 - Good balance of creativity & coherence (typical: 0.8–0.95)
- Top-k sampling: samples only from the top-k most probable tokens.
 - Limits the candidate pool (typical: 20–50)

Rule of thumb: Use low T + top-p for factual QA. Use higher T or top-k for creative writing or brainstorming.

Q28. How do you evaluate an agent's performance beyond just "did it complete the task?"

Task Success	Correctness	Efficiency	Robustness	Safety	User Satisfaction
Did it achieve the goal?	Is the final answer correct and grounded?	How many steps, tool calls, tokens used?	Handles edge cases, messy inputs?	No harmful, biased, or unsafe outputs?	User feedback, ratings, NPS, thumbs up/down?

Bottom line: Use a mix of automatic metrics + human evaluation.

Q29. What is context engineering and how is it different from prompt engineering?

Prompt Engineering	Context Engineering
Focus on how you instruct the model. Crafting prompts, examples, formats, system messages. Works on the "instructions" layer.	Focus on what context you provide. Selecting, ordering, compressing, and structuring the right information. Works on the "information" layer.

Takeaway: Good context > perfect prompt.

Q30. How do you handle LLM API failures and rate limits in a production application?

Control	How
Retry with exponential backoff	Handle 429/5xx with jittered backoff
Circuit breaker	Stop calling after repeated failures; fail fast, then recover
Fallback models	Use a smaller / cheaper model as fallback
Queue & throttle	Rate-limit requests, queue when needed

Control	How
Graceful degradation	Return cached results, partial answers, or helpful error messages
Monitoring & alerts	Track errors, latency, rate-limit hits

- Best practices: idempotent requests
 - Timeouts
 - Request collapsing
 - Caching
 - User-friendly errors

Key takeaway: *Design for scale, design for failure, and optimize for users.*

SECTION 4 — SCENARIO-BASED (Q31 – Q35)

Q31. You shipped a RAG feature last week. This week, user satisfaction dropped 20%. The model and index have not changed. What do you investigate?

- Query distribution shift (sources / APIs)
- Upstream data changes (processing / pipelines)
- Retrieval quality drop (recall / precision)
- Chunking / prompt / formatting errors
- Latency / timeouts / errors
- UI / UX or answer-presentation issues
- User request inspected (new users / org?)

How	
Compare analytics before vs. after	
Log traces, and errors	
Sample bad queries and compare retrieved context vs. answers	
Check user feedback / abandon rate	
Run evals on a held-out set	

Goal: Find what changed in reality, not in code.

Q32. An enterprise client wants to deploy your LLM application on-premises because of data privacy requirements. What changes?

Area	Key Changes
Model	Local inference: use on-prem LLMs (e.g. Llama, Mistral) via vLLM / cLM
Data storage	All embeddings, data, and logs remain on-prem
Self-hosted retrieval	Vector DB (Qdrant/Weaviate), reranker, monitoring, observability
Networking	Private VPC / VLAN, firewalls, no outbound internet (or allowlist only)
Security & compliance	SSO / SAML integration, encryption at rest / in transit, audit logs
DevOps	On-prem CI/CD, model updates, backups, disaster recovery

Trade-off: Higher infra cost, more responsibility, potentially smaller models.

Q33. You are given a 500-page legal document and asked to build a Q&A system that lawyers can use. What do you build?

1. Ingestion	2. Chunking	3. Indexing	4. Retrieval	5. Generation	6. UX for Lawyers
Convert PDF → text; OCR if scanned; extract metadata (sections, clauses, page numbers)	Semantic chunking; ~300–800 tokens with overlap; preserve clause structure	Create embeddings; store in vector DB; store metadata (section, page, doc id)	Top-k semantic search; metadata filters (dates, doc id); rerank results	Grounded answer w/ citations; include page/section references; "I don't know" if insufficient context	Clean Q&A interface; citations link to sections/pages; follow-up questions; export / share answers

Nice-to-have: Version & updates; highlighted citations; follow-up questions; search within a section.

Great systems are built by diagnosing the right problem and optimizing continuously.

Q34. Your agent is sending duplicate emails to customers. How do you diagnose and fix it?

Diagnose:

- Check logs: tool calls, inputs, outputs, timestamps
- Look for retries / timeouts causing re-execution
- Verify idempotency: does the email tool prevent duplicates?
- Check agent / state management: is the agent remembering it already sent the email?
- Review prompts & instructions for unclear guidance
- Check external systems (webhook retries, queue)

Fix:

- Make email tool idempotent (require a unique request id / idempotency key)
- Write confirmations to durable storage (DB) and check before sending
- Add guardrails in prompt: "Do not send the same email twice"
- Use a human-in-the-loop for critical actions
- Add dedupe window (e.g. don't send same email within N hours)
- Add monitoring & alerts for duplicate actions

Q35. How would you reduce the cost of a RAG system that currently spends \$8,000/month on LLM API calls?

1. Reduce Calls	2. Improve Retrieval	3. Optimize Prompts	4. Model Strategy	5. Operational
Add semantic cache for similar queries; batch or dedupe requests where possible	Better chunking & fewer irrelevant tokens; metadata filters to narrow top-k (e.g. 10 → 5)	Shorter prompts (summarize/compress context); remove verbose examples / templates	Use smaller / cheaper models for most tasks; distill or fine-tune model per use case; real-time not required for all flows	Monitor cost per feature / stage; set token budgets; use streaming to cut off failing/long calls early

Track \$/query continuously and optimize the most expensive components first.

Great systems are built by diagnosing the right problem and optimizing continuously.

SECTION 5 — CODING (Q36 – Q38)

Q36. Write a function that calls the OpenAI API with exponential backoff on rate-limit errors.

```
import openai
import time
import random
from openai import OpenAI
from openai.error import RateLimitError, APIError, Timeout

client = OpenAI()

def call_openai_with_backoff(messages, model="gpt-4o-mini",
                             max_retries=6, base_delay=1.0, max_delay=60.0):
    attempt = 0
    while True:
        try:
            response = client.chat.completions.create(
                model=model,
                messages=messages,
                timeout=30,
            )
            return response
        except (RateLimitError, Timeout) as e:
            attempt += 1
            if attempt > max_retries:
                raise RuntimeError(f"Max retries ({max_retries}) exceeded. Last error: {e}")
            delay = min(max_delay, base_delay * (2 ** (attempt - 1)))
            jitter = random.uniform(0, delay * 0.1)
            print(f"Rate limit. Retry in {delay + jitter:.2f}s (attempt {attempt})")
            time.sleep(delay + jitter)
        except APIError as e:
            # Non-rate-limit API errors: don't retry
            raise e
```

- Retries on RateLimitError and Timeout
- Exponential backoff: $\text{base_delay} \times 2^{(\text{attempt}-1)}$
- Adds jitter after base_delay ranges
- Stops after max_retries

Q37. Write a minimal RAG pipeline that takes a list of text documents and a query.

```
from typing import List
from openai import OpenAI
import numpy as np

client = OpenAI()
EMBED_MODEL = "text-embedding-3-small"
CHAT_MODEL = "gpt-4o-mini"
TOP_K = 4

# 1) Embed documents
def embed(texts: List[str]) -> np.ndarray:
    res = client.embeddings.create(model=EMBED_MODEL, input=texts)
    return np.array([d.embedding for d in res.data])
```



```

# 2) Retrieve top-k by cosine similarity
def retrieve(query, doc_embeddings, docs, top_k=TOP_K):
    q_emb = embed([query])[0]
    sims = doc_embeddings @ q_emb / (
        np.linalg.norm(doc_embeddings, axis=1) * np.linalg.norm(q_emb) + 1e-8)
    top_idx = np.argsort(-sims)[:top_k]
    return [docs[i] for i in top_idx]

# 3) Generate answer
def rag_query(query: str, docs: List[str]) -> str:
    doc_embs = embed(docs)
    context_docs = retrieve(query, doc_embs, docs)
    context = "\n\n".join(context_docs)
    system = ("You are a helpful assistant. Answer using ONLY the context. "
             "If the answer isn't in the context, say 'I don't know'.")
    resp = client.chat.completions.create(
        model=CHAT_MODEL,
        messages=[
            {"role": "system", "content": system},
            {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {query}"},
        ],
        temperature=0,
    )
    return resp.choices[0].message.content

```

Minimal by design: No external index, in-memory search, easy to extend to a persistent vector DB (Qdrant, Pinecone) and reranking.

Q38. Write a function definition for an LLM that searches a database and handles argument validation.

```

from typing import Optional, Literal, Dict, Any
from pydantic import BaseModel, Field, validator

class SearchDBArgs(BaseModel):
    table: str = Field(..., description="Table to search, e.g. 'customers'")
    query: str = Field(..., description="Search text or SQL WHERE-style condition")
    limit: int = Field(10, ge=1, le=50)
    filters: Optional[Dict[str, Any]] = None
    sort_by: Optional[str] = None

    @validator("table")
    def table_must_be_allowed(cls, v):
        allowed = {"customers", "orders", "products"}
        if v not in allowed:
            raise ValueError(f"table must be one of {allowed}")
        return v

def search_database(args: SearchDBArgs) -> dict:
    # ... validated args are safe to use in a parameterized query
    return {"rows": [], "count": 0}

# Example tool schema passed to the LLM
tool_schema = {
    "name": "search_database",
    "parameters": SearchDBArgs.schema(),
}

```

- Key points: validates inputs with Pydantic
- Prevents SQL injection via parameterized queries
- Restricts table to an allowlist
- Bounds limit (1–50)
- Returns structured JSON the LLM can read

SECTION 6 — BEHAVIORAL (Q39 – Q42)

Q39. Tell me about a time you improved the performance of an AI system you owned.

Situation:

Our RAG chatbot had low answer accuracy (~62%) and users often said answers were incomplete.

Action:

- Analyzed failures: low retrieval recall + poor chunking
- Switched from fixed 500-token chunks to hierarchical chunking (sections + paragraphs)
- Added a cross-encoder reranker (bge-reranker-base)
- Improved query rewriting with MyBE + multi-query
- Added citation requirement in the prompt
- Built an eval set and tracked RAGAS metrics

Result:

- Answer correctness 62% → 86%
- Hallucination rate 28% → 9%
- User satisfaction 3.2/5 → 4.5/5
- Support tickets tied to wrong answers ↓ 60%

Key takeaway: Data + retrieval quality + evaluation = biggest lever.

Q40. Describe a time you had to make a tradeoff between model quality and cost in production.

Context:

A summarization feature was costing ~\$6K/month on GPT-4. Usage was high, but we needed to optimize.

Option A (High Quality) GPT-4	Option B (Cost Optimized) GPT-3.5-turbo + Rerank
Best quality, handles edge cases, high cost, slower	5× cheaper, faster, slightly lower fluency, needs prompt tuning

What I did:

- Used GPT-3.5-turbo with an improved prompt + length control
- Added a low-cost filter (LLM-as-judge) to catch bad outputs
- Escalated only complex inputs to GPT-4 (10–15% of traffic)

Outcome:

- Cost reduced by ~72% (\$6K → \$1.7K/month)
- Quality impact minimal (GPT-4.3 → 4.1/5)
- Latency improved ~35%

Lesson: Optimize for the 80% first, and route the rest.

Q41. Tell me about a production AI incident you managed. What happened and what did you change?

Incident:

Users reported the chatbot giving confident but incorrect answers after a data-source update.

Impact:

- ~18% of answers contained outdated information
- Trust and CSAT dropped
- Support volume increased

Root Cause:

Our ingestion job partially failed, so the index had a mix of old and new documents. We had no freshness checks.

What I did:

- Rolled back to last known-good index
- Added data freshness validation + alerts
- Implemented source rebuild (versioning), not incremental patch
- Added a "last updated" date to answers
- Built a canary and end-to-end detail regression checks

Outcome:

- Issue resolved in ~2 hours
- No repeat incidents since implementing safeguards
- CSAT recovered within a week

What I changed long-term:

Better monitoring, data validation, and evals in CI/CD for our RAG pipeline.

Q42. How do you stay current in this field?

- Daily Learning (30 mins): read research papers / arXiv summaries; follow key blogs (OpenAI, Anthropic, LangChain, Vercel AI, Hugging Face, Pinecone)
- Hands-on Practice: build small projects and experiment with new tools / models; reproduce open-source projects and learn by doing
- Community: engage in Discords / r/MachineLearning / r/LocalLLaMA; contribute to open source & share learnings
- Newsletters: subscribe to TLDR AI, Import AI, Latent Space, The Batch; skim weekly to catch trends and new releases
- Continuous Reflection: keep a learning journal / changelog; run postmortems on wins & failures
- Conferences & Talks: watch talks from NeurIPS, ICML, ACL, PyData, etc.; attend virtual meetups and webinars

Mindset: "Stay curious, ship often, and keep learning in public."

Behavioral is about impact, ownership, and continuous growth. Show how you think, build, learn, and make things better.

SECTION 7 — MODEL SELECTION & INFRASTRUCTURE (Q43 – Q58)

Q43. When would you choose a 13B open-weight model over a frontier API model like GPT-5?

- Data privacy / compliance (no data leaves your infra)
- Cost at scale (predictable, one-time infra cost vs. usage fees)
- Offline / air-gapped environments
- Customizability (fine-tune, modify, add tools/features)
- Lower latency for local use cases
- Good enough quality for your task (domain-specific)
- You have MLOps capability to run and maintain it

Q44. What is quantization and how does it affect LLM inference?

Quantization reduces the precision of model weights (e.g. FP32 → INT8/INT4) to save memory and speed up inference.

FP16 (16-bit) → INT4 (4-bit)

- Effects:
 - Memory usage ↓ (4–8× less)
 - Throughput / faster inference ↑
 - Cost ↓ (smaller GPUs / more tokens per \$)
 - Slight quality drop (INT4 > INT8 > FP16 in loss)

Q45. What is the difference between latency and throughput in LLM serving, and how do you optimize each?

Latency (time to first token / response time)	Throughput (tokens/sec or requests/sec)
Affects single-user experience. Optimize by: faster model, shorter context, quantization, KV-cache, speculative decoding, continuous batching, better infra (GPU/TPU)	Affects total capacity. Optimize by: batching, larger context parallelism, tensor / pipeline parallelism, kv-cache reuse, vLLM / TGI, autoscaling

Q46. What is speculative decoding and when should you use it?

Speculative decoding uses a small draft model to generate multiple tokens ahead, which the large model verifies in parallel.

- Use when:
 - You need lower latency for long outputs
 - You have a good small draft model (e.g. 1–7B)
 - You can trade a bit more compute for big speedups (1.5–3×)

Q47. Explain the difference between HNSW and IVF indexing in vector databases. When does each perform better?

HNSW (Hierarchical Navigable Small World)	IVF (Inverted File Index)
Graph-based index; excellent recall; low latency for search; high memory usage; good for dynamic datasets; best for high-recall + low-latency requirements (top-k search)	Cluster-based (k-means + inverted lists); lower memory usage; faster build, better for large scale; recall depends on nprobe; best for very large datasets (millions–billions) where memory is limited and slight latency trade-off is acceptable

Rule of thumb: HNSW for quality & speed, IVF for scale & memory efficiency.

Q48. What is prompt compression and how does it reduce cost without sacrificing quality?

Reduce prompt tokens while keeping the essential information.

11. Summarization of long docs / chat history
12. Extractive compression (keep key sentences/facts)
13. Remove redundant / low-importance tokens
14. Use semantic filtering + structured representations (tables, JSON)

Fewer input tokens → lower cost + faster latency while preserving the signal the model needs.

Q49. How do you handle PII (Personally Identifiable Information) in an LLM application?

- Detect: use PII detection (regex + NER models)
- Minimize: don't collect what you don't need
- Mask / Redact: replace PII with tags (e.g. [NAME], [EMAIL])
- Encrypt: at rest and in transit
- Isolate: private VPC / on-prem / dedicated deployment
- Audit: log access, retain policies, user consent
- Delete: data-retention policies + right to be forgotten

Q50. What is DSPy and how does it differ from traditional prompt engineering?

DSPy (Declarative Self-improving Prompting): a framework to program LM behaviors as modules with signatures (inputs/outputs) and automatically optimize prompts using feedback/data.

Traditional Prompt Engineering	DSPy
Manual prompts; ad-hoc iteration; hard to scale / compose	Declarative (signatures & modules); auto-optimization (teleprompting); reusable, composable

Q51. What is a knowledge graph and when does GraphRAG outperform standard vector RAG?

Knowledge Graph (KG): entities (nodes) + relationships (edges) with structured, explicit semantics.

- GraphRAG outperforms when:
 - Queries need multi-hop reasoning across relationships
 - Data is highly structured (people, orgs, products, law, medical)
 - Need explanations with provenance via connected facts
 - Handling large schemas / evolving relationships

Q52. What is the difference between zero-shot, one-shot, and few-shot prompting?

Zero-shot	One-shot	Few-shot
No examples. Just instruction + input. Generalization relies on model priors.	One example + input. Sets the pattern.	Several examples + input. Better performance on complex tasks.

Q53. How would you implement A/B testing for LLM prompt changes in production?

15. Define metric(s): e.g. task success, CSAT, conversion, latency, cost
16. Create variants: Prompt A (control) vs. Prompt B (treatment)
17. Traffic split: randomly assign % of users/requests
18. Log everything: prompts, inputs, outputs, latency, tokens, cost
19. Evaluate with stats: significance tests, guardrail metrics
20. Rollout: gradual (e.g. 10% → 50% → 100%) with kill switch
21. Monitor: continually watch dashboard + user feedback

Q54. What is model distillation and when is it used?

Model distillation trains a smaller "student" model to mimic the outputs of a larger "teacher" model.

Teacher (Model) —knowledge transfer→ Student (Model, smaller)

- Use when:
 - You need lower latency / cost
 - Deploy on edge or smaller GPUs
 - Protect IP (replace proprietary model with a smaller one)
 - Maintain quality with good compression

Q55. Explain the concept of structured outputs and why they matter for production systems.

Structured outputs ensure the model returns data in a predefined format (JSON, schema, function call, XML).

- Why it matters:
 - Reliable parsing & automation

- Less post-processing & fewer errors
- Enforces contracts for downstream systems
- Improves safety and consistency

Q56. What is retrieval with metadata filtering and when is it essential?

Use metadata (e.g. tenant_id, doc_type, date, language) to filter candidates before/while retrieving.

- Essential when:
 - Multi-tenant isolation (user/org boundaries)
 - Legal / compliance constraints (region, sensitivity)
 - Time-aware info (latest docs only)
 - Domain-specific subsets (product, department)

Q57. What is the attention sink phenomenon and how does it affect very long context inference?

Attention sink: the model over-attends to early tokens (or special tokens) and under-weights the middle of long contexts.

Impact: Important info in the middle gets ignored → worse answers.

- Mitigation:
 - Put critical info at the beginning or end
 - Use chunking + retrieval
 - Use models with better long-context handling
 - Use attention-sink tokens (e.g. "sink" tokens / repeat summary)

Q58. How do you version and manage prompts in production?

- Store prompts in a versioned repository (Git or Prompt DB)
- Use semantic versioning (e.g. v1.2.0)
- Track metadata: description, owner, created_at, model, variables
- Use templates with variables & defaults
- A/B test and promote: dev → staging → prod
- Rollback capability
- Monitoring: track performance per prompt version

Dev (v1.3.0) → Staging (v1.3.0) → Prod (v1.2.0)

SECTION 8 — ADDITIONAL INTERMEDIATE (Q59 – Q74)

Q59. What is the difference between LangChain and LangGraph? When do you use each?

- LangChain: toolkit for building chains, prompts, retrieval, tools. Great for simple to moderately complex flows.
- LangGraph: graph-based orchestration for complex, stateful, cyclical agent workflows.

Use LangChain for rapid prototyping. Use LangGraph for complex agents with loops, branching, persistence.

Q60. What is the "needle in a haystack" test and what does it evaluate?

Evaluates long-context retrieval of a specific fact ("needle") placed in a large irrelevant context ("haystack"). Measures recall across increasing context lengths.

Q61. What is RLHF and why did DPO largely replace PPO-based RLHF for most post-training?

- RLHF: trains model using human feedback (rankings) to align behavior.
- DPO (Direct Preference Optimization): simpler, more stable, avoids reward model + PPO complexity, lower compute, fewer hyperparameters.

Q62. What is chain-of-thought prompting and when does it help?

Encourages the model to reason step-by-step before answering. Helps on complex reasoning, math, logic, multi-hop problems.

Q63. How does an embedding model differ from a generation model architecturally?

- Embedding model: encoder-only (e.g. BERT-style). Outputs dense vectors.
- Generation model: decoder-only or encoder-decoder. Autoregressive token generation.

Q64. What is the difference between streaming and batch inference for LLMs?

- Streaming: returns tokens incrementally; lower perceived latency; holds connection open.
- Batch: processes many requests together; higher throughput; better GPU utilization; higher first-token latency.

Q65. What is a system prompt and what are the security implications of putting sensitive logic in it?

System prompt sets model behavior and rules. If leaked via jailbreak / prompt injection, attackers can bypass guardrails. Don't put secrets, keys, or critical logic there.

Q66. What is a reranker and how does it differ from a retriever in a RAG pipeline?

- Retriever: finds relevant docs (recall).

- **Reranker:** reorders candidates by relevance (using cross-encoder or LLM). Improves precision.

Q67. What is few-shot prompting via retrieval (also called RAG-based few-shot)?

Retrieve example Q&A pairs similar to the user query and include them in the prompt as in-context examples.

Q68. What is constitutional AI and how does Anthropic use it to align Claude?

A framework with principles and critiques where the model critiques and revises its own responses. Anthropic trains Claude using this approach alongside other alignment methods.

Q69. How do you debug an agent that is producing incorrect final answers despite having the correct information in its tool outputs?

- Inspect intermediate steps and tool I/O.
- Verify prompt instructions and output format.
- Check reasoning/aggregation step.
- Add unit tests, traces, and step-level evals.

Q70. What is the difference between document Q&A and conversational RAG, and how do they require different architectures?

- Document Q&A: single-turn over a static corpus.
- Conversational RAG: multi-turn with chat history, follow-ups, coreference.

Conversational RAG needs memory, history-aware retrieval, and query rewriting.

Q71. What are the key differences between OpenAI Agents SDK, LangGraph, and CrewAI for production agent development?

- OpenAI Agents SDK: opinionated, tight with OpenAI models/tools; fastest path to production.
- LangGraph: flexible, low-level control; best for custom complex workflows.
- CrewAI: role-based multi-agent teams; good for collaborative task execution.

Q72. What is the chain-of-verification (CoVe) technique and when is it valuable?

Model generates multiple reasoning paths or answers, then verifies / cross-checks them to arrive at the most consistent answer. Valuable for high-stakes or error-prone tasks.

Q73. How do you handle a situation where a user's query spans multiple documents that contradict each other?

- Retrieve widely, then rerank.
- Surface both sides with sources.

- Ask clarifying question if needed.
- Use citation + confidence, let user decide.

Q74. What is agent memory staleness and how do you manage it?

Staleness = memory contains outdated facts or state. Manage by: TTLs, refresh policies, source re-checks, versioned memory, summarization with timestamps.

Master the fundamentals. Apply them in context. Ship reliable AI systems.

SECTION 9 — SYSTEM DESIGN SHORT-ANSWER (Q75 – Q80)

Q75. How would you design a GenAI system to handle traffic spikes without overwhelming the model provider?

- Queue & rate limit: use a request queue (e.g. Redis/SQS) and enforce concurrency & rate limits.
- Backpressure: return "queued" status / ETA for non-urgent requests.
- Load shedding: drop or defer low-priority traffic during spikes.
- Caching: semantic cache for answers; prompt + response cache where possible.
- Model tiering: route to smaller/cheaper/faster models when under pressure.
- Autoscale workers: scale out stateless workers that pull from the queue.
- Circuit breaker: open when error rate/latency crosses thresholds; fail fast or downgrade.
- Pre-compute & warm: pre-embed, pre-rank, and keep hot caches warm.
- Multi-provider fallback: failover to secondary provider if primary is throttling/down.

Q76. What metrics would you put on a GenAI product dashboard for a non-technical stakeholder?

- User Impact: active users, queries/conversations, tasks completed.
- Success Rate: % successful answers, task completion rate.
- User Satisfaction: thumbs up/down %, CSAT.
- Time to Value: avg. time to first good answer.
- Cost: total cost, cost per 1K queries.
- Quality (high level): accuracy/helpfulness trend (using evals or feedback).
- Reliability: uptime, error rate.
- Adoption: feature usage, repeat users.
- Top Queries/Topics: what users ask most.

Q77. How do you prevent your RAG system from returning stale information?

- Fresh data pipeline: ingest on change (CDC, webhooks) or frequent scheduled syncs.
- Document versioning: store versions with timestamps and source URLs.
- Recency-aware retrieval: boost recent docs; filter by date when relevant.
- TTLs: set expiration for cached results and embeddings if content is time-sensitive.
- Source citation: always show source + last updated date.
- Validation step: for critical facts, verify against authoritative APIs/databases.
- Monitoring: track data freshness log (source update → index update).

Q78. When would you use pgvector instead of a dedicated vector database like Qdrant?

Use pgvector when:

- Your data already lives in Postgres.
- You need strong consistency + ACID transactions with vectors.
- Dataset is small-to-medium (up to ~10M vectors depending on hardware).

- You need simple ops (one database to manage).
- You require SQL joins + filters with vector search.

Use a dedicated DB (e.g. Qdrant) when:

- You have very large scale (10M+ to billions of vectors).
- You need advanced ANN performance, sharding, replication.
- You need hybrid dense/sparse, payload filtering at scale, or multi-tenancy features.

Q79. What is the difference between an LLM gateway and an orchestration framework?

	LLM Gateway	Orchestration Framework
Purpose	Manage access to LLMs.	Build and manage complex LLM workflows/agents.
What it does	Routing, load balancing, rate limiting, caching, logging, cost control, model fallbacks.	Chains, agents, tools, memory, state, retries, conditional logic, human-in-the-loop.
Scope	Closer to infrastructure / platform.	Application-level logic and flow.
Examples	LiteLLM, Portkey, AI Gateway, Kong AI Gateway	LangGraph, LlamaIndex, CrewAI, Microsoft Autogen

In practice: Use a gateway for reliability/cost/control. Use an orchestration framework to build the app.

Q80. How do you build a GenAI feature that needs to degrade gracefully when the LLM API is unavailable?

- Detect early: health checks and circuit breaker.
- Fallback hierarchy:
 - Use cached answer (semantic/response cache).
 - Use a smaller/cheaper local or alternate model.
 - Use rules/templates/FAQ for simple queries.
 - Show helpful message with option to retry.
- Partial functionality: keep non-LLM features working (search, browse, uploads).
- Queue for later: accept request and process when service is back.
- Communicate clearly: inform users about degraded mode and expected behavior.
- Log & alert: track failures and user impact.

Design for scale. Measure what matters. Degrade gracefully.